

Distribute Coins Approach

The key to this problem is to **stop thinking about individual coin moves** and instead think:

For every subtree, how many extra coins does it have, or how many coins does it need?

This is a classic **DFS + postorder + balance/excess flow** problem.

1. The main observation

Every node must finally have exactly:

1 coin

Suppose we look at a subtree.

For example:



The root has:

3 coins

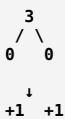
But it only needs:

1 coin

So:

extra = 3 - 1 = 2

Those 2 extra coins must leave the root:



Answer = 2 moves.

2. Define the important concept: balance

For every subtree, we calculate:

balance = total coins in subtree - total nodes in subtree

But we don't need to explicitly count coins and nodes.

Using DFS:

```

balance = node->val - 1
          + leftBalance
          + rightBalance
  
```

Meaning

If:

balance > 0

The subtree has extra coins.

Example:

balance = +3

It must send 3 coins to its parent.

If:

```
balance < 0
```

The subtree needs coins.

Example:

```
balance = -2
```

It needs 2 coins from its parent.

If:

```
balance = 0
```

The subtree is perfectly balanced.

3. Why do we use postorder DFS?

Consider:



We cannot determine what the root should do until we know:

- whether the left subtree needs coins
- whether the right subtree has extra coins

So we process:

```

left subtree
  ↓
right subtree
  ↓
current node

```

That is exactly **postorder traversal**.

4. Let's understand Example 2 step by step

Tree:



We start from the leaves.

Left node

```
3
```

It needs only 1 coin.

```
balance = 3 - 1 = +2
```

So this subtree has:

```
+2 extra coins
```

It must send 2 coins upward.

Number of moves:

```
2
```

Visualization:



```

    1 1
    0

```

Right node

```
0
```

It needs one coin.

```
balance = 0 - 1 = -1
```

So it needs:

```
1 coin
```

Its parent will eventually have to send one coin.

Moves contributed:

```
1
```

Root

Root value:

```
0
```

Left balance:

```
+2
```

Right balance:

```
-1
```

Therefore:

```
balance = node.val - 1 + left + right
```

```
balance = 0 - 1 + 2 + (-1)
```

```
balance = 0
```

Perfect. The entire tree has exactly enough coins.

Total moves:

```
|leftBalance| + |rightBalance|
```

```
= |2| + |-1|
```

```
= 2 + 1
```

```
= 3
```

Answer:

```
3
```

5. Why do we add `abs(balance)` to the answer?

This is the most important part.

Consider a child subtree with:

```
balance = +3
```

That means the subtree has 3 extra coins.

The **only connection** between this subtree and the rest of the tree is this edge:



Therefore, those 3 coins must cross this edge.

Each coin crossing an edge costs:

1 move

So:

3 coins × 1 edge = 3 moves

Thus:

```
C++
moves += abs(childBalance);
```

Similarly:

balance = -2

means 2 coins must cross the edge into that subtree:



Again:

2 moves

Therefore:

```
C++
moves += abs(balance);
```

6. The DFS idea

For every node:

Step 1

Get balance from the left subtree:

```
C++
int left = dfs(root->left);
```

Step 2

Get balance from the right subtree:

```
C++
int right = dfs(root->right);
```

Step 3

All excess/deficit coins from children must cross their connecting edges.

So:

```
C++
moves += abs(left);
moves += abs(right);
```

Step 4

Calculate the balance that this whole subtree sends to its parent:

```
C++
return root->val - 1 + left + right;
```

7. The code

```
C++
class Solution {
public:
    int moves = 0;

    int dfs(TreeNode* root) {
        if (root == nullptr) {
            return 0;
        }

        // Get excess/deficit from left subtree
        int leftBalance = dfs(root->left);

        // Get excess/deficit from right subtree
        int rightBalance = dfs(root->right);

        // Coins that must cross the edges
        moves += abs(leftBalance);
        moves += abs(rightBalance);

        // Return this subtree's excess/deficit
        return root->val - 1 + leftBalance + rightBalance;
    }

    int distributeCoins(TreeNode* root) {
        dfs(root);
        return moves;
    }
};
```

8. Let's dry run a slightly more complex example

Consider:



Total nodes = 5, so total coins = 5.

Node 3

```
balance = 3 - 1
         = +2
```

It sends 2 coins upward.

```
moves = 2
```

Node 0 next to it

```
balance = 0 - 1
         = -1
```

It needs 1 coin.

```
moves = 2 + 1 = 3
```

Their parent



Its balance:

```
balance = 0 - 1 + 2 - 1
         = 0
```

This entire subtree is balanced.

No coins need to cross upward.

Right child of root

```
0
```

```
balance = -1
```

It needs 1 coin.

```
moves = 3 + 1
        = 4
```

Root

```
balance = 1 - 1 + 0 - 1
          = -1
```

Wait—this example actually has only 4 coins, so it violates the problem's condition that total coins equal total nodes. In a valid input, the root's final balance will always be:

```
0
```

That is another useful property.

9. The formula to remember

For every node:

```
left = dfs(root->left)
right = dfs(root->right)

moves += |left| + |right|

return root->val - 1 + left + right
```

Or conceptually:

DFS returns

```
extra coins → +ve
needed coins → -ve
```

10. The intuition in one sentence

Each subtree tells its parent how many coins it has extra or how many coins it needs, and the absolute value of that number is exactly how many coin moves must cross the edge connecting them.

This pattern appears in many tree problems:

- distribute resources in a tree
- balance loads between nodes
- move items through tree edges
- subtree surplus/deficit problems
- DFS returning information from child to parent

So the pattern you should recognize here is:

```
POSTORDER DFS
+
SUBTREE BALANCE
+
COUNT FLOW ACROSS EDGES
```

The time complexity is:

```
O(n)
```

because every node is visited once.

Space complexity:

$O(h)$

for the recursion stack, where h is the tree height.

Sure. Here is the complete **C++ solution**:

```
C++
class Solution {
public:
    int moves = 0;

    int dfs(TreeNode* root) {
        // Base case
        if (root == nullptr) {
            return 0;
        }

        // Get balance from left subtree
        int leftBalance = dfs(root->left);

        // Get balance from right subtree
        int rightBalance = dfs(root->right);

        // Count coins moving across edges
        moves += abs(leftBalance);
        moves += abs(rightBalance);

        // Return excess (+) or deficit (-) coins
        return root->val - 1 + leftBalance + rightBalance;
    }

    int distributeCoins(TreeNode* root) {
        dfs(root);
        return moves;
    }
};
```

The most important line

```
C++
return root->val - 1 + leftBalance + rightBalance;
```

It means:

```
current node coins
- 1 coin needed by itself
+ extra/needed coins from left subtree
+ extra/needed coins from right subtree
```

And:

```
C++
moves += abs(leftBalance);
moves += abs(rightBalance);
```

counts how many coins must cross the edges.

For example:

```
leftBalance = +2
```

→ left subtree sends **2 coins** to the current node → **2 moves**

```
rightBalance = -3
```

→ current node sends **3 coins** to the right subtree → **3 moves**

So the DFS is essentially returning:

```
positive → I have extra coins
negative → I need coins
zero      → My subtree is balanced
```

ChatGPT can make mistakes. Check important info.